

WHIM — Explicación del Código

RESUMEN EJECUTIVO

WHIM es una app de descubrimiento de lugares en Valencia. El usuario elige una categoría (comida, ocio nocturno, compras...), un mood (cita, rápido, celebración...) y un nivel de precio. La IA busca los mejores 5 lugares y genera un análisis honesto de cada uno basado en reseñas reales.

Stack:

- **Backend:** Python + FastAPI, desplegado en Railway
- **Frontend:** React Native + Expo (web y móvil desde el mismo código)
- **IA:** Gemini 2.5 Flash (Google Vertex AI)
- **Datos:** Google Places API (lugares + reseñas), Yelp, TripAdvisor
- **BD:** SQLite en local, PostgreSQL en producción + Supabase (auth y realtime)

Flujo principal de la IA (3-5 segundos):

1. Google Places devuelve hasta 60 candidatos con reseñas inline
2. Se filtran y rankean por rating, distancia y precio
3. Se enriquecen con reseñas de Yelp y TripAdvisor en paralelo
4. Gemini analiza las reseñas y genera: tagline, por qué encaja con tu mood, pros, contras y veredicto
5. Los resultados llegan progresivamente al móvil (polling) o al web (SSE streaming)

Módulos clave:

- `pipeline.py` → orquesta todo el proceso de IA
- `google_places_service.py` → fuente principal de datos
- `brain_service.py` → capa de IA con fallbacks (Gemini → OpenRouter → Groq → Ollama)
- `category_flow.py` → define todas las categorías, subcategorías y moods
- `api.ts` (frontend) → cliente HTTP con polling progresivo
- `useAppState.ts` (frontend) → estado global de la app

Estructura del Proyecto

```
DataProject3/
|
|_ main/
|   |_ backend/
|       |_ main.py           ← Arranque FastAPI (CORS, routers)
|       |_ config.py        ← Variables de entorno (Supabase, Google,
Gemini)
|       |_ database.py      ← Conexión y queries a la base de datos
|       |_ auth.py          ← Autenticación
|       |_ models/
```

└─ schemas.py	← Modelos de datos (Pydantic)
└─ routers/	← ENDPOINTS (lo que usa el frontend)
└─ recommend.py	← ★ Recomendaciones IA (batch / SSE / polling)
└─ search.py	← Búsqueda de lugares
└─ places.py	← Detalle de un lugar
└─ reports.py	← Reportes ciudadanos
└─ votes.py	← Votos en reportes
└─ bookmarks.py	← Lugares guardados
└─ preferences.py	← Preferencias de usuario
└─ services/	← LÓGICA DE NEGOCIO
└─ google_places_service.py	← ★ API Google Places
└─ brain_service.py	← ★ IA (Gemini)
└─ vector_service.py	← Embeddings
└─ place_persistence_service.py	← Guardado en BD
└─ nominatim_service.py	← Geocodificación
└─ overpass_service.py	← Datos OSM
└─ cache_service.py	← Caché en memoria
└─ recommendation/	← ★★ MOTOR DE IA
└─ pipeline.py	← Motor principal
└─ category_flow.py	← Lógica de categorías
└─ tools.py	← Herramientas del agente
└─ frontend/	
└─ app/	← React Native + Expo
└─ main/supabase/migrations/	← SQL de la base de datos

BACKEND

main.py — Arranque de FastAPI

Tres bloques: lifespan (arranque), middleware, routers.

Lifespan: context manager asíncrono que se ejecuta al arrancar el servidor. Llama a `init_db()` para inicializar la base de datos. Si falla, lo loguea en `startup.log` pero no revienta el servidor.

Middleware:

- `CatchAllMiddleware` — middleware de error genérico, actualmente **comentado**. Tiene lógica para saltarse los endpoints de streaming (`/stream`) y no bufferizar la respuesta SSE.
- `CORSMiddleware` — permite cualquier origen (`allow_origins=["*"]`). `allow_credentials=False` porque con `"*"` no se pueden enviar cookies (limitación del estándar CORS).

Routers registrados: health, auth, recommend, votes, categories, places, events, reports, bookmarks, search, brain, photos, preferences, compare, deals, reservations, interactions, internal.

config.py — Variables de entorno

Carga inteligente de .env: detecta automáticamente si está en un entorno gestionado (Cloud Run, Railway, App Engine) mirando variables específicas de cada plataforma (`K_SERVICE`, `RAILWAY_ENVIRONMENT_NAME`, `GAE_ENV`). En producción no carga el `.env` (las variables las inyecta la plataforma). En local sí las carga.

Claves de IA relevantes:

- `GOOGLE_MAPS_API_KEY` → Google Places (búsqueda de lugares)
- `GOOGLE_GENAI_API_KEY` / `GEMINI_API_KEY` → intercambiables, usa el que esté disponible
- `BRAIN_PROVIDER` → motor de IA: "gemini" (por defecto), también soporta `openrouter`, `groq`, `ollama`
- `OPENROUTER_API_KEY` / `GROQ_API_KEY` → alternativas a Gemini
- `OLLAMA_URL` + `OLLAMA_MODEL` → para modelos locales sin coste

Parámetros de la app:

- `MAX_DISTANCE_KM = 5.0` → radio máximo de búsqueda
- `DEFAULT_MAP_RADIUS_M = 10000` → radio del mapa por defecto

database.py — Capa de compatibilidad de base de datos

El mismo código funciona con **SQLite en local** y **PostgreSQL en producción** sin cambiar nada.

Dos modos:

- `DATABASE_URL` vacío → SQLite (`temp_local.db`)
- `DATABASE_URL` con valor → PostgreSQL (Cloud SQL / Supabase)

PostgresCompatConnection: wrapper sobre `asyncpg` que imita la interfaz de `aiosqlite`. La función `_translate_sql()` convierte automáticamente los `?` a `$1`, `$2...` antes de ejecutar en Postgres. Así el resto del código siempre escribe con `?` y funciona en ambos.

get_db(): context manager que se usa en todos los routers para obtener una conexión:

```
async with get_db() as db:
    await db.execute("SELECT ...", (param,))
```

Seeds de datos: categorías, subcategorías, moods y tipos de reporte están hardcodedos. Al arrancar por primera vez, si las tablas están vacías, los inserta automáticamente.

Tablas principales:

- `places` → lugares guardados en BD local
- `community_reports` → reportes ciudadanos
- `categories` / `category_subcategories` / `category_moods` → árbol de categorías
- `saved_items` → bookmarks de usuario
- `item_votes` → votos en reportes
- `user_preferences` → configuración por usuario

Nota: Supabase no se usa aquí. Este archivo es para la BD local/Cloud SQL. Supabase se usa directamente desde el frontend con su SDK propio.

auth.py — Autenticación

Dos funciones:

get_optional_user(): devuelve siempre "local-user". Es un **mock** — en producción debería verificar el JWT de Firebase, pero esa lógica no está implementada en el backend. La autenticación real está delegada al frontend con Supabase Auth.

get_voter_id(): genera un ID anónimo estable para votar en reportes sin cuenta. Hashea IP + User-Agent con SHA-256 y coge los primeros 16 caracteres. Permite que el mismo dispositivo siempre tenga el mismo ID sin guardar nada en BD.

models/schemas.py — Contratos de datos (Pydantic)

RecommendRequest — lo que manda el frontend al pedir recomendaciones:

- **parent_category** + **subcategory** → ej. "food" + "sushi"
- **mood** → ej. "date", "quick", "party"
- **priceLevel** → 1-4 (opcional)
- **lat** / **lng** → ubicación del usuario
- **language** → idioma de la respuesta ("es" por defecto)

PlaceResult — lo que devuelve la IA por cada lugar. Campos generados por Gemini:

- **tagline** → frase corta del lugar
- **why** → por qué se recomienda para ese mood
- **pros** / **cons** → puntos fuertes/débiles
- **verdict** → veredicto final
- **bestReviewQuote** → mejor frase extraída de reseñas
- **reviewQualityScore** → calidad de las reseñas (0-1)

Campos de Google Places: **rating**, **address**, **photoUrl**, **distanceM**, etc.

RecommendResponse → { **top**: PlaceResult[] }, máximo 5 resultados.

ROUTERS

routers/recommend.py — ★ Puerta de entrada a la IA

Define **3 formas** de pedir recomendaciones al mismo motor:

POST /recommend — Batch (todo de golpe) El cliente espera bloqueado hasta que la IA termina y devuelve los 5 mejores resultados de una vez.

POST /recommend/stream — SSE (streaming en tiempo real) Usa Server-Sent Events. La IA va enviando cada resultado conforme lo termina. Ideal para web con **ReadableStream**.

POST /recommend/start + GET /recommend/poll/{job_id} — Polling progresivo El cliente lanza el job y recibe un `job_id`. Luego hace polling periódico. Lo usa la app móvil porque los streams SSE son difíciles de manejar en React Native.

El parámetro `after=N` en el poll evita re-descargar resultados ya vistos.

El store `_jobs` es un diccionario en memoria (no Redis). Los jobs se limpian solos a los 5 minutos (`_JOB_TTL = 300`).

`MAX_TOP_RESULTS = 5` se aplica en todos los modos.

routers/search.py — Búsqueda universal

GET /search/universal Combina 3 fuentes en paralelo con `asyncio.gather()`:

1. Google Places → resultados con fotos y ratings reales
2. Overpass (OSM) → POIs de OpenStreetMap
3. BD local → places/events/reports guardados

Antes de buscar, pasa la query por `ask_brain()` (Gemini) para interpretar la intención. Ej: "quiero algo barato para cenar" → `query="restaurante", category="food"`. Tiene doble caché: una para la query original y otra para la query interpretada.

Los resultados de Google y OSM se guardan en BD en background sin bloquear la respuesta.

GET /search/autocomplete Llama a Nominatim (OpenStreetMap) para autocompletar direcciones. Devuelve máximo 5 sugerencias.

GET /search/live Igual que `/universal` pero en streaming — envía resultados conforme llegan de cada proveedor usando `asyncio.as_completed()`.

routers/places.py — Detalle de lugar

GET /places/nearby — devuelve el mapa completo: busca en Google Places + OSM en paralelo, y saca reportes y eventos de la BD local. Mezcla todo como `map_items`.

GET /places/{id}/take — llama a `enrich_place_result()` del pipeline de IA para generar el análisis completo de un lugar concreto (tagline, pros, cons, verdict...). Es el mismo enriquecimiento del pipeline de recomendaciones pero para un lugar individual.

routers/reports.py, votes.py, bookmarks.py, preferences.py — CRUD social

reports.py:

- **GET /reports/nearby** → reportes en un radio (bounding box aproximado)
- **POST /reports** → crear reporte con tipo, título, lat/lng y duración
- **POST /reports/{id}/confirm** → votar si el reporte es real (1) o falso (-1), con upsert
- **DELETE /reports/{id}** → solo el creador puede borrar el suyo

votes.py:

- `POST /votes` → like/dislike con upsert. Voto 0 = eliminar voto
- `GET /votes/{id}` → conteo de likes/dislikes + tu voto
- `POST /votes/batch` → lo mismo para múltiples items a la vez (eficiente para el mapa)

bookmarks.py:

- `GET /bookmarks` → lista con JOIN a places/events/reports para traer título y foto
- `POST /bookmarks` → guardar con `ON CONFLICT DO NOTHING` (idempotente)
- `DELETE /bookmarks/{id}` → eliminar

preferences.py:

- `GET/PATCH/PUT /preferences` → leer y actualizar preferencias (radio, categorías favoritas, tema, idioma, estilo de mapa...)
- `favorite_cats` se serializa como JSON string en BD y se deserializa al leer
- Tiene rutas duplicadas `/preferences` y `/preferences/me` por compatibilidad

SERVICES — MOTOR DE IA

`services/recommendation/category_flow.py` — Árbol de categorías

Define toda la estructura de navegación de la app: qué categorías existen, qué subcategorías tienen, qué moods, y cómo se comporta la búsqueda.

FLOW_DEFINITIONS: diccionario con una entrada por categoría. Cada categoría tiene:

- `search_mode` → `"guided_ranked"` (IA recomienda), `"event_list"` (lista eventos), `"report_only"`
- `requires_price` → si el usuario debe elegir nivel de precio
- `fallback_radius_m / max_radius_m` → radio de búsqueda
- `provider_types` → tipos de Google Places a buscar (ej. `["restaurant", "cafe"]`)
- `subcategories + moods` → opciones que ve el usuario en la app

Tres tipos de categorías:

1. **Guided ranked** (food, nightlife, shopping...) → la IA busca y rankea
2. **Event list** (event, market, music) → muestra eventos de la BD, sin IA
3. **Report only** (report) → solo reportes ciudadanos

Funciones clave:

- `get_flow_definition(category_id)` → devuelve la definición de una categoría
- `build_flow_payload(...)` → construye el payload completo para el frontend
- `merge_category_row()` → combina datos hardcoded con datos de la BD (la BD puede sobrescribir label, radio, etc.)
- `_normalize_food_options()` → lógica especial para "food": asegura que "Tapas" esté en subcategorías y no en moods

`services/recommendation/tools.py` — Herramientas de búsqueda

Conecta el pipeline de IA con las APIs externas.

search_places() — el buscador principal. Construye una query inteligente y devuelve hasta 60 resultados con reseñas incluidas:

1. **_build_query()** → construye la query según categoría+subcategoría (ej. "sushi restaurant")
2. **_apply_budget_to_query()** → añade "barato economico" o "premium exclusivo" según precio
3. **_apply_mood_to_query()** → añade contexto del mood (ej. "rapido para llevar" si mood=quick)
4. **_search_runtime_options()** → decide radio, si filtrar por **open_now**, y si ordenar por distancia
5. Filtra resultados: excluye tiendas retail cuando la categoría no es shopping

SEARCH_RESULT_TARGET = 60 → pide 60 resultados a Google, el pipeline luego filtra y se queda con los mejores 5.

fetch_all_reviews() — lanza 3 peticiones en paralelo con **asyncio.gather()**:

- Google Places details (timeout 5.5s)
- Yelp reviews (timeout 12s)
- TripAdvisor reviews (timeout 22s)

Cada una tiene su propio timeout independiente. Los resultados se cachean 30 minutos.

get_place_details() — wrapper simple sobre Google Places para obtener teléfono, foto y reseñas de un lugar concreto.

haversine() — calcula distancia real en metros entre dos coordenadas.

services/google_places_service.py — ★ API Google Places

Wrapper sobre la **Google Places API (New)**. Es la fuente principal de datos de lugares, fotos y reseñas.

Decisiones de diseño importantes (del propio código):

- Las reseñas se piden **inline** en la búsqueda → evita N llamadas extra de detalle
- No se usa **includedType** estricto → deja que la IA de Google decida la relevancia
- Google devuelve máximo 5 reseñas por lugar (límite duro de la API)
- **reviewSummary** (GA Mayo 2025) es un resumen generado por Gemini de TODAS las reseñas del lugar, mucho más rico que las 5 individuales

search_places() — búsqueda principal Hace peticiones en paralelo en múltiples idiomas (**asyncio.gather**) para maximizar la cobertura de reseñas. Por ejemplo para **language="es"** lanza búsquedas en ["es", "en", "ca"] simultáneamente y luego fusiona los resultados deduplicando reseñas por fingerprint (**autor + rating + texto**).

Soporta filtros opcionales: **price_levels**, **open_now**, **rank_preference** (DISTANCE o RELEVANCE). Cachea resultados 5 minutos.

El campo **field_mask** en la cabecera **X-Goog-FieldMask** controla exactamente qué campos devuelve Google (y por tanto qué se cobra). Incluye: id, nombre, dirección, coordenadas, rating, fotos, tipos, horarios, reseñas y reviewSummary.

`get_place_details()` — detalle de un lugar concreto Igual que search pero para un `place_id` específico. Lanza hasta 5 peticiones en paralelo en distintos idiomas para maximizar reseñas. Opcionalmente enriquece con Yelp (`include_yelp=True`). Cachea 24 horas.

`get_photo_bytes()` — proxy de fotos Descarga la foto binaria de Google y la cachea 1 hora. El frontend llama al endpoint `/photos/google/{photo_name}` del backend, que a su vez llama a esta función. Así la API key nunca se expone al cliente.

`_photo_proxy_url()` — en lugar de devolver la URL directa de Google, devuelve una URL relativa al backend (`/photos/google/...`). El backend actúa de proxy.

`services/brain_service.py` — ★ IA pluggable (Gemini + fallbacks)

Capa de abstracción sobre LLMs. Permite cambiar de proveedor sin tocar el resto del código.

Proveedores soportados (en orden de prioridad):

1. `gemini` → Vertex AI (Gemini 2.0 Flash) — producción
2. `openrouter` → meta-llama/llama-4-maverick — alternativa gratuita
3. `groq` → llama-3.3-70b — muy rápido, gratuito con límites
4. `ollama` → modelo local (por defecto `qwen2.5-coder:7b`) — sin coste, sin internet

`ask_brain()` — función principal. Recorre la cadena de proveedores (`_provider_chain()`) hasta que uno funciona. Si todos fallan, devuelve un fallback que simplemente devuelve la query original sin interpretar.

SYSTEM_PROMPT — le dice al LLM que es el asistente de WHIM y que debe devolver siempre un JSON con:

```
{
  "query": "término de búsqueda limpio",
  "category": "food|health|shopping|...|null",
  "intent": "search|report|info|chat",
  "response": "respuesta natural al usuario"
}
```

`_parse_brain_response()` — extrae el JSON de la respuesta del LLM buscando el primer `{` y el último `}`. Robusto ante texto extra antes/después del JSON.

`ask_brain_stream()` — versión streaming de Gemini, devuelve chunks de texto conforme los genera el modelo.

`_resolve_vertex_project()` — detecta el proyecto de Google Cloud desde la variable de entorno o desde las credenciales ADC (Application Default Credentials). Se cachea en memoria para no repetir la detección.

`services/vector_service.py` — Embeddings semánticos

Genera embeddings de texto usando el modelo `text-embedding-004` de Google (vía el cliente Gemini del pipeline). Incluye `cosine_similarity()` para comparar vectores.

Uso: comparar la intención del usuario (mood en lenguaje natural) con los nombres/descripciones de lugares para un ranking semántico más preciso. Actualmente es auxiliar — el pipeline principal usa scoring heurístico, no embeddings.

services/place_persistence_service.py — Guardado en BD

Una sola función relevante: `upsert_provider_places()`. Recibe una lista de lugares de cualquier proveedor (Google, OSM) y los inserta en la tabla `places` de la BD local con `ON CONFLICT DO UPDATE`. Así la BD local se va enriqueciendo en background con cada búsqueda sin bloquear al usuario.

`_canonical_place_row()` normaliza el formato de cada proveedor a la estructura de la tabla `places`.

services/nominatim_service.py — Geocodificación (OSM)

Wrapper sobre Nominatim (OpenStreetMap). Dos funciones:

- `geocode(query)` → texto a coordenadas (usado en `/search/autocomplete`)
- `reverse_geocode(lat, lng)` → coordenadas a dirección

Implementa rate limiting manual (1 req/s) con un lock asíncrono, respetando los términos de uso de Nominatim. Incluye `User-Agent` identificativo obligatorio.

services/overpass_service.py — Datos OSM (Overpass API)

Busca POIs en OpenStreetMap usando la Overpass API. Gratuita, sin API key.

`CATEGORY_TO_OSM` mapea categorías de la app a filtros OSM (ej. `"food"` → `[amenity~"restaurant|cafe|bar|..."]`). Si se pasa una `query` de texto, añade un filtro `["name"~"query",i]` para buscar por nombre.

Complementa a Google Places con datos de OSM que Google puede no tener (pequeños negocios locales, espacios públicos, etc.).

services/cache_service.py — Caché en memoria

Caché en memoria simple con TTL. Implementado como un diccionario Python `_CACHE: Dict[str, (expires_at, value)]`.

- `cache_get(key)` → devuelve el valor si no ha expirado, `None` si expiró o no existe
- `cache_set(key, value, ttl=300)` → guarda con TTL en segundos (por defecto 5 min)
- `cache_purge()` → limpia entradas expiradas

Es **por proceso** — si hay múltiples instancias del backend (ej. en Railway con escalado horizontal), cada una tiene su propia caché. No es Redis, no es compartida. Suficiente para el volumen actual.

services/recommendation/pipeline.py — ★★ Motor principal de recomendaciones

El archivo más importante del proyecto. Orquesta todo el proceso de recomendación de principio a fin.

Flujo completo (batch ~3-5s, stream primer resultado ~3s):

```
A search_places()      → Google Places, hasta 60 candidatos + reseñas inline
B semantic_filter()     → descarta baja calidad, rankea por rating × log(reseñas) +
precio + distancia
C' smart_fetch()        → fetcha reseñas completas (Google + Yelp + TripAdvisor) en
paralelo
D resolve_mood()        → DETERMINÍSTICO: mood → preferencias estructuradas (sin
LLM)
E LLM + live data       → 2 batches LLM (3+2) + live data TODO EN PARALELO
F fallback              → si el LLM falla, usa reviewSummary directamente (nunca
devuelve vacío)
```

Paso B — `_semantic_filter()`

Filtra y rankea los candidatos. Tres niveles:

1. **Filtro básico:** descarta lugares con rating < 3.5 o distancia > 8km.
2. **Filtro de presupuesto:** prioriza coincidencia exacta de precio, luego cercana, luego desconocida, luego lejana. Nunca deja la lista vacía.
3. **Búsqueda semántica (opcional):** si el mood es una frase larga (>15 chars), genera embeddings del mood y de cada lugar y rankea por similitud coseno. Para moods cortos como "date" usa el ranking heurístico normal.

La función de scoring combina: `rating × log10(reseñas) - penalización_distancia - penalización_precio + boost_semántico + boost_mood`.

Paso D — `_resolve_mood()`

Convierte el mood en preferencias estructuradas **sin llamar al LLM**. Es un diccionario hardcodeado (`_MOOD_MAP`) con ~70 moods mapeados a:

```
{"prefer_quiet": bool, "prefer_fast": bool, "prefer_formal": bool,
"prefer_outdoor": bool, "vibe_keywords": [...]}
```

Esto es mucho más rápido y predecible que pedirle al LLM que interprete el mood cada vez.

Paso E — LLM en paralelo

Los 5 candidatos se dividen en 2 batches (3+2) y se procesan en paralelo junto con las peticiones de live data:

```
await asyncio.gather(
    _llm_batch(batch1, ...),    # 3 lugares
```

```

    _llm_batch(batch2, ...),    # 2 lugares
    get_live_data(lugar1),
    get_live_data(lugar2),
    ...
)

```

`_build_llm_prompts()` construye el prompt con instrucciones explícitas:

- Usar las reseñas individuales como fuente primaria, `reviewSummary` solo como confirmación
- Ser honesto, incluir negativos, no usar tono de marketing
- El campo `why` debe explicar por qué encaja con el tipo, mood y presupuesto pedido
- Devolver JSON estricto con: `tagline`, `why`, `pros`, `cons`, `verdict`, `tags`, `best_quote`, `quality_score`

El modelo usado es `gemini-2.5-flash` (configurable con `GEMINI_MODEL`).

Paso F — Fallback sin LLM

Si Gemini falla, `_enrich_fallback()` construye el resultado usando análisis heurístico de las reseñas:

- `_fallback_review_signals()` detecta patrones positivos/negativos en el texto de las reseñas usando listas de keywords (`_POSITIVE_THEMES`, `_NEGATIVE_THEMES`, `_PRACTICAL_CAUTIONS`)
- Genera pros, cons y verdict sin LLM
- Nunca devuelve un resultado vacío

`recommend()` vs `recommend_stream()`

- `recommend()` → espera a que todos los pasos terminen y devuelve los 5 resultados de golpe
- `recommend_stream()` → procesa en batches y hace `yield` de cada resultado conforme termina. El primer resultado llega en ~3s, los demás van llegando de 2 en 2.

`enrich_place_result()` — versión para un lugar individual (usado en `/places/{id}/take`). Mismo proceso pero para un solo lugar, con caché de 1 hora.

FRONTEND

Estructura general

```

frontend/
├── app/
│   ├── _layout.tsx      ← Root layout (providers, navegación)
│   ├── index.tsx        ← Pantalla de inicio / splash
│   ├── (tabs)/          ← Navegación principal (tab bar)
│   │   ├── index.tsx    ← Mapa principal
│   │   ├── explore.tsx  ← Explorar categorías
│   │   ├── report.tsx   ← Crear reporte ciudadano
│   │   └── profile.tsx  ← Perfil de usuario
│   └── (flow)/          ← Flujo de recomendación (wizard)

```

├─ category.tsx	← Paso 1: elegir categoría
├─ mood.tsx	← Paso 2: elegir mood
├─ price.tsx	← Paso 3: elegir precio
├─ results-map.tsx	← ★ Resultados con mapa
├─ details.tsx	← Detalle de un resultado
├─ (modals)/	← Pantallas modales
├─ place-details.tsx	← Detalle de lugar (desde mapa)
├─ settings.tsx	← Configuración
├─ login.tsx / register.tsx	
├─ saved-items.tsx / my-reports.tsx	
├─ services/	
├─ api.ts	← ★ Cliente HTTP del backend
├─ supabase.ts	← Cliente Supabase (auth)
├─ mapService.ts	← Helpers del mapa
├─ preferences.ts	← Gestión de preferencias
├─ hooks/	
├─ useAppState.ts	← ★ Estado global (Context)
├─ useLocation.ts	← Geolocalización
├─ useAuth.ts	← Autenticación Firebase/Supabase
├─ useRealtime.ts	← Suscripciones Supabase realtime
├─ components/	
├─ map/	← Componentes del mapa
├─ Map.tsx	← Selector plataforma
├─ Map.native.tsx	← react-native-maps (móvil)
├─ Map.web.tsx	← Leaflet (web)
├─ whim/	← Componentes del flujo IA
├─ WhimSelectionTree.tsx	← Árbol de selección categoría/mood
├─ WhimLoadingScreen.tsx	← Pantalla de carga con animación
├─ WhimResultsPreviewCard.tsx	← Preview de resultado
├─ RestaurantCard.tsx	← Tarjeta de resultado IA
├─ LiveDataAddon.tsx	← Widget de datos en tiempo real
├─ ...	
├─ constants/	
├─ design.ts	← Sistema de diseño (colores, tipografía, espaciado)
├─ theme.ts	← Tema claro/oscuro
├─ locales/	
├─ es.json	← Traducciones español
├─ en.json	← Traducciones inglés

services/api.ts — ★ Cliente HTTP

El archivo más grande del frontend (38KB). Contiene todas las llamadas al backend.

Autodetección de URL: `getBaseUrl()` detecta el entorno automáticamente:

- Si hay `EXPO_PUBLIC_BACKEND_URL` en el `.env`, lo usa
- Si está en Railway (`*.railway.app`), usa la URL hardcodeda de producción
- Si está en local, usa `localhost:8000`

Funciones de recomendación (las más importantes):

- `recommendRestaurants()` → llama a `POST /recommend` (batch, espera todo)
- `recommendRestaurantsStream()` → llama a `POST /recommend/start` + polling a `GET /recommend/poll/{job_id}`. Devuelve resultados conforme llegan, llamando a un callback `onResult` por cada uno.

Fallback de ubicación: Si la geolocalización falla, usa Valencia centro (39.4699, -0.3763) como fallback.

hooks/useAppState.ts — ★ Estado global

Context de React que centraliza todo el estado de la app. Persiste en `AsyncStorage` (excepto `nearbyItems` y `results` que son transitorios).

Estado del flujo de recomendación:

- `parentCategory` → categoría principal (ej. "food")
- `category` → subcategoría (ej. "sushi")
- `mood` → mood elegido (ej. "date")
- `priceLevel` → 1/2/3
- `results` → los 5 resultados de la IA

Estado del mapa:

- `mapRegion` → región visible del mapa
- `nearbyItems` → items del mapa (lugares, reportes, eventos)
- `mapPreferences` → estilo de mapa, radio, tema, idioma

Los setters están memoizados con `useCallback`. Al cambiar de categoría, resetea automáticamente los pasos siguientes (mood, precio, resultados).

app/(flow)/results-map.tsx — ★ Pantalla de resultados

La pantalla más compleja del frontend. Muestra el mapa con los 5 resultados de la IA y permite navegar entre ellos.

Flujo de carga:

1. Llama a `recommendRestaurantsStream()` con polling
2. Muestra `WhimLoadingScreen` con animación mientras carga
3. Conforme llegan resultados, los va añadiendo a la lista con `LayoutAnimation`
4. Cada resultado aparece como pin en el mapa y como `RestaurantCard` en el bottom sheet

Estados: "loading" → "streaming" (primeros resultados) → "success" (todos) / "error"

Mapa dual (web vs nativo)

`Map.tsx` es un selector de plataforma que importa:

- `Map.native.tsx` en iOS/Android → usa `react-native-maps` (Google Maps)
- `Map.web.tsx` en navegador → usa `Leaflet` (OpenStreetMap, sin coste)

Esta separación es necesaria porque `react-native-maps` no funciona en web.

Internacionalización

`locales/es.json` y `locales/en.json` contienen todas las cadenas de texto. El idioma se detecta del sistema o se puede forzar en preferencias. `utils/i18n.ts` expone la función `t()` para traducir.

BASE DE DATOS (Supabase Migrations)

Las migraciones se aplican en orden en Supabase:

`000_full_reset.sql` — Limpieza total y creación de tablas base. Punto de partida limpio.

`001_item_votes_text_id.sql` — Cambia el tipo de `item_id` en `item_votes` de UUID a TEXT para soportar IDs de Google Places (que son strings como `ChIJ...`).

`002_category_flow_preferences_and_report_fixes.sql` — Añade tablas de preferencias de usuario, subcategorías y moods de categorías, y corrige la tabla de reportes.

`003_expanded_categories_and_subcategories.sql` — Expande el catálogo de categorías y subcategorías con todas las que aparecen en `category_flow.py`.

El esquema de Supabase es paralelo al de `database.py` — Supabase se usa desde el frontend (auth, realtime), la BD local del backend es independiente.

Documento generado el 2026-05-05. WHIM — Discovering the real city, brutally honest.